

Sistema de Gestión de Inventario

Contexto importante:

- “Una **pequeña empresa** necesita llevar el control de su inventario de productos.”

→ Teniendo esto en cuenta, considero para la toma de decisiones:

- Tiempo de desarrollo reducido.
- Recursos a dedicar reducidos.
- El sistema no escalará en tamaño inmediatamente.
- El uso y mantenimiento del sistema debe ser claro, amigable y económico.

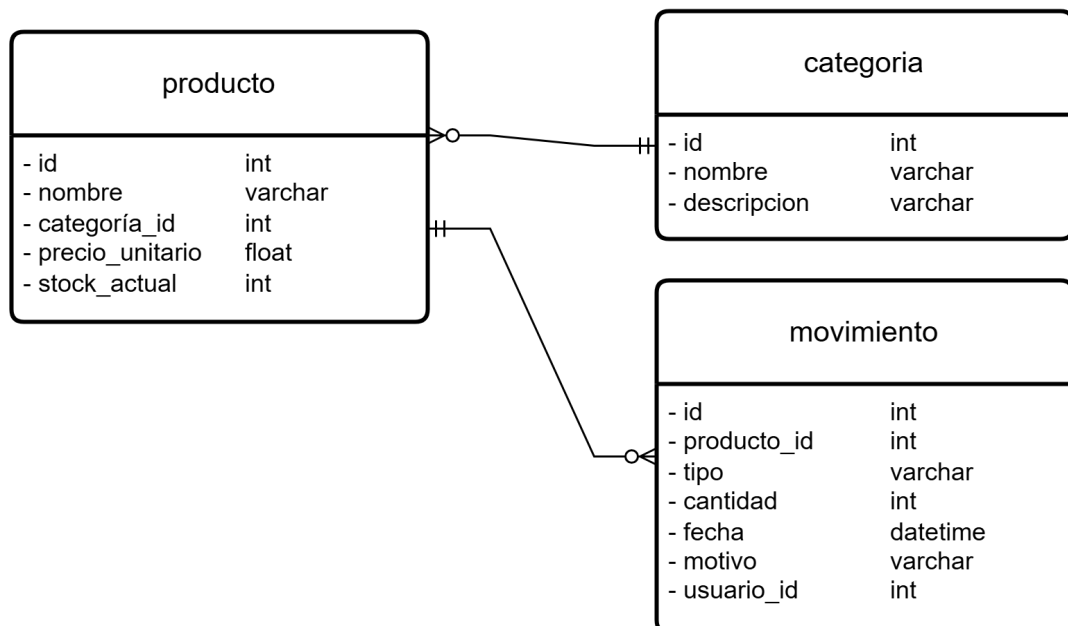
Base de datos

Diseño del modelo

Lo primero es diseñar correctamente las entidades y sus relaciones para evitar futuros errores e inconsistencias en la lógica de los datos. Para ello, adoptando las buenas prácticas aprendidas en la materia “Bases de Datos”, comienzo realizando un *Diagrama Entidad-Relación*.

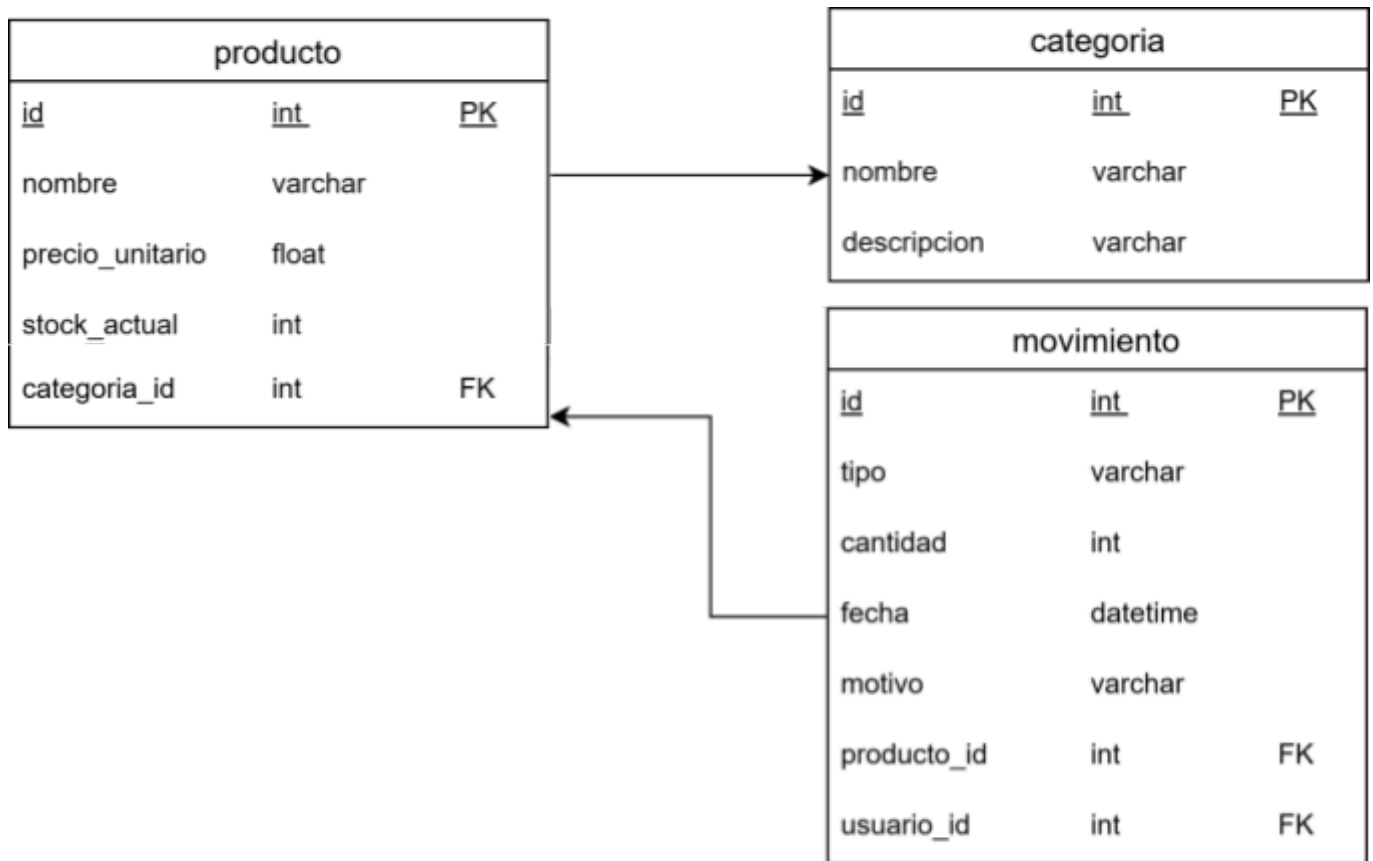
Este paso me parece importante para ver claramente lo que estoy modelando, además de comprender e incluso identificar requerimientos importantes desde el principio. Esto no me demanda mucho tiempo pero ahorrará tiempos de corrección en un futuro.

Diagrama Entidad-Relación:



En base a lo anterior, paso al *modelo físico de los datos* que se implementará en la base de datos.

MFD:



Podemos ver que en “movimiento” se contempla la existencia de una futura tabla “usuario”, por eso la FK usuario_id, que serviría para implementar la autenticación y la autorización basada en roles.

Utilicé modelos ORM para definir las tablas de la base de datos. Los mismos se encuentran en el repositorio en la carpeta backend/app/models

¿Qué tipo de base de datos elegirías y por qué?

En primer lugar, usaría una base de datos relacional como PostgreSQL, con la que he trabajado. Esto es básicamente por la naturaleza de los datos a manejar. Un inventario es altamente estructurado, además de que es crítico cumplir con las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) para evitar inconsistencias que podrían comprometer la operatividad de la empresa y la toma de decisiones. Estas son características que cumplen las bases de datos relacionales.

¿Usarías un ORM? ¿Qué ventajas y desventajas ves en este caso?

Sí usaría un ORM, por varias razones. Entre las ventajas encuentro que, considerando importante un tiempo de desarrollo reducido, el ORM acelera las tareas. Además, sabemos que al abstraer las consultas permite cambiar o interactuar con distintas bases de datos sin cambiar significativamente el código, y previene inyecciones SQL de forma nativa.

Sin embargo, lo haría prestando especial atención en hacer correctamente las consultas para que sean eficientes, resolviendo el problema N+1. Por ejemplo, con SQLAlchemy, usaría *joinedload* y *selectinload*.

¿Cómo garantizarías que el stock no quede negativo?

A nivel de base de datos, esto lo garantizaría con un check constraint del tipo: CHECK (stock_actual >= 0) en la tabla de producto. También agregaría un chequeo previo, a nivel backend cuando se revisen las reglas de negocio, y a nivel frontend, validando la operación antes de ejecutarla para mejorar la experiencia de usuario y evitar desde la primera línea flujos de información incorrectos.

¿Cómo conviven estas herramientas dentro del stack tecnológico de un proyecto? ¿Cuáles usarías y por qué? (Por ejemplo: ¿ORM junto con migraciones? ¿Cliente directo en algún caso? ¿Cómo interactúan con el resto de la arquitectura?)

En primer lugar está la base de datos, que es donde están realmente almacenados los datos y donde se garantiza que reglas duras, como que el stock no sea menor a cero, se cumplan sí o sí. Después el ORM actúa como un traductor entre Python y la base de datos en SQL, facilitando su manejo y previniendo vulnerabilidades.

Por otra parte, sí usaría ORM con migraciones, como SQLAlchemy con Alembic, que facilita que la base de datos evolucione con el proyecto. Alembic toma los cambios que se hacen en los modelos de Python y genera un script automático para aplicarlos en la base de datos de forma segura.

Sí usaría también SQL directo, en caso de que por ejemplo escale la cantidad de datos y necesite hacer queries con muchísimos JOINS, agrupaciones y subconsultas para hacer un reporte. Esto porque el ORM a veces genera código SQL genérico que no es el más eficiente.

Todo esto interactúa con el resto de la arquitectura cumpliendo su rol dentro de la misma, ya que la arquitectura idealmente está dividida de manera modular y cada parte tiene su responsabilidad. Primero, el cliente a través del frontend envía un JSON con los datos del movimiento, y el framework de la API recibe la petición en un Endpoint. Allí usa herramientas como Pydantic para validar que el JSON tenga la forma correcta. Si está todo bien, el servicio recibe esos datos validados. En base a la acción a ejecutar, el ORM toma esas órdenes del servicio y las traduce a consultas SQL que viajan a la base de datos. La base de datos ejecuta la operación, confirma que todo está bien y le avisa al ORM. Finalmente, el framework le devuelve un "200 OK" al frontend.

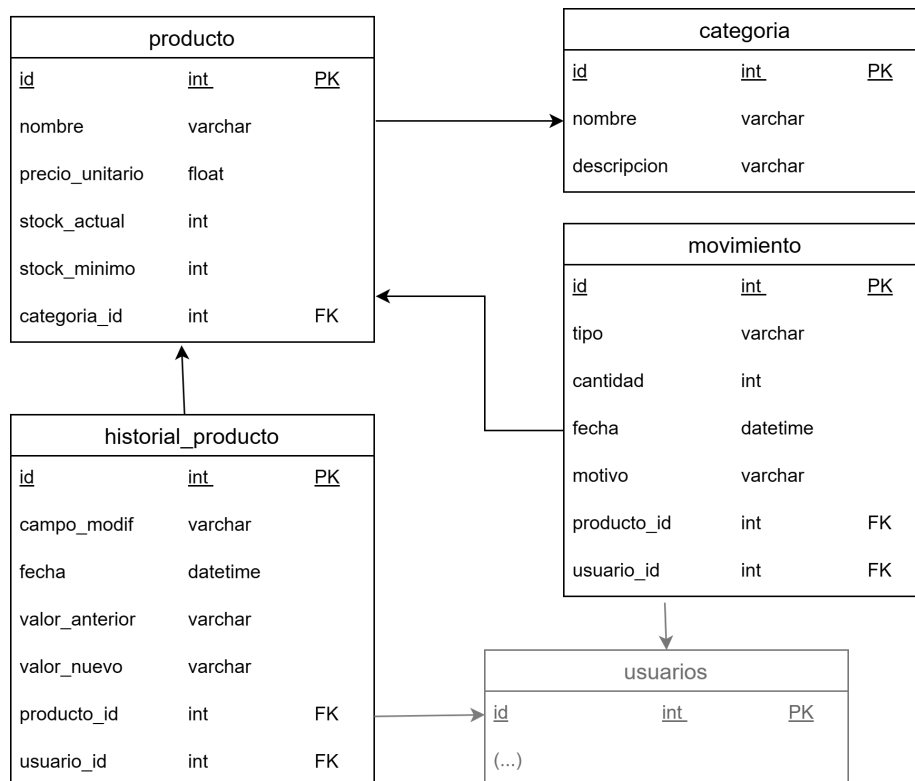
Todo esto ocurre por ejemplo dentro de contenedores como Docker, asegurando que el ORM se pueda conectar a la base de datos mediante las credenciales correctas pasadas por variables de entorno.

Historial de cambios

Si alguien edita el nombre o precio de un producto, ¿cómo guardarías el valor anterior? ¿Guardarías esa auditoría en la misma tabla, en una tabla aparte o en otro sistema?

Para esto se suelen usar tablas de auditoría, por lo que crearía una tabla aparte llamada "historial_producto" que esté asociada sí o sí a uno y solo un producto, que guarde campos como "producto_id" (FK), "campo_modificado", "valor_anterior", "valor_nuevo", "fecha" y "usuario_id" (FK).

En este caso, como es una empresa pequeña creo que lo mejor sería tener esta nueva tabla dentro de la misma base de datos, donde también estaría la tabla de "usuarios", pero entiendo que en sistemas grandes esta auditoría debería ir a un sistema externo para no sobrecargar la base principal.



Para pensar (no implementar)

Si el sistema crece, ¿cómo lo encarás?

- El inventario pasa a tener 500 000 productos con miles de movimientos diarios.
- Se necesita saber el stock histórico de un producto en cualquier fecha pasada.

Si el sistema creciera de esta manera, sería muy importante optimizar el diseño de la base de datos y las consultas más frecuentes, prestar atención al máximo a las buenas prácticas ya que muchas cosas comienzan a ralentizarse.

Mantendría separadas las tablas de productos y movimientos de inventario, utilizando la tabla de movimientos como fuente de verdad para reconstruir el historial completo de stock. Para mejorar el rendimiento, agregaría índices sobre campos muy consultados, especialmente id de producto, fecha de movimiento, categoría, y posiblemente combinaciones como (producto_id, fecha).

Además, para soportar correctamente consultas como el stock histórico de un producto en cualquier fecha pasada, evitaría depender únicamente del stock actual almacenado en la tabla de productos. En cambio, el stock podría reconstruirse a partir de los movimientos acumulados hasta esa fecha.

Si el volumen siguiera creciendo, podría evaluarse particionar la tabla de movimientos por fecha, o generar snapshots periódicos de stock.

Por otra parte, también consideraría agregar estrategias de paginación para las interacciones, tanto por performance como por experiencia de usuario.

Backend

Decisiones de arquitectura

¿Qué lenguaje y framework usarías?

Usaría Python con FastAPI para el backend. Elegí FastAPI porque permite desarrollar APIs REST de forma rápida y mantenible, tiene muy buen soporte para validación de datos mediante Pydantic y genera documentación automática, lo que facilita probar y entender los endpoints.

Además, me parece una buena elección para este proyecto porque permite estructurar el sistema en capas de manera clara sin agregar demasiada complejidad. También consideré que FastAPI tiene buen rendimiento, tipado moderno y una curva de desarrollo muy cómoda para integrar autenticación JWT, ORM y validaciones.

¿Cómo organizarías el proyecto en carpetas? *Prestá especial atención a cómo pensás la orquestación del proyecto: cómo se conectan las capas entre sí (rutas, servicios, repositorios, modelos), y cómo esa estructura facilita mantener y escalar el sistema.*

Organizaría el proyecto backend utilizando una arquitectura en capas para separar responsabilidades y facilitar el mantenimiento del sistema a medida que crece. La idea es desacoplar la lógica HTTP, la lógica de negocio y el acceso a datos, evitando que toda la lógica quede concentrada en los endpoints.

El flujo inicia con el cliente que hace una request HTTP, la cual accede a la ruta correspondiente, que redirige a un método en los servicios que aplican la lógica de negocio, para finalmente llegar a los repositorios que acceden a la base de datos ORM/DB. Entonces:

- routes/: definen los endpoints y manejan la interacción HTTP.
- services/: contienen las reglas de negocio y validaciones del sistema.
- repositories/: encapsulan el acceso a base de datos usando SQLAlchemy. → (para ese caso no lo implementé por simplicidad, y dejé esta lógica en services)
- models/: representan las tablas de la base de datos mediante ORM.
- schemas/: definen los modelos Pydantic para validar requests y responses.
- db/: centraliza la configuración de la conexión y sesiones de base de datos.
- core/: contiene configuración global y utilidades de seguridad/autenticación.

Por ejemplo, una regla de negocio como validar stock disponible antes de registrar una salida debería resolverse en la capa de services y no directamente en el endpoint o en el acceso a datos.

Para este proyecto mantuve las consultas ORM simples dentro de la capa de services para evitar complejidad innecesaria con la capa repositories. Si el sistema creciera o aparecieran queries más complejas, separaría el acceso a datos en repositories dedicados.

Al dividir el proyecto en módulos separados con responsabilidades bien definidas y lo más atómicas posible, estoy aplicando buenas prácticas de la ingeniería de software. Este enfoque es crucial para la mantenibilidad y escalabilidad del sistema, ya que simplifica tanto el desarrollo como las pruebas al permitirnos enfocarnos específicamente en lo que se necesita corregir, añadir o modificar. Además, optimiza mucho la colaboración y el trabajo en equipo.

Por otra parte, a la parte de frontend la manejaría en una carpeta aparte, contigua a backend, dentro del directorio principal del proyecto. En este directorio principal es donde además se realizaría la configuración de docker compose para poder levantar todos los servicios en un contenedor.

¿Dónde y cómo configurarías las reglas de negocio (ej: stock mínimo, tipos de movimientos válidos)?

Las reglas de negocio las implementaría principalmente en la capa de services, ya que esa capa debería centralizar la lógica funcional del sistema. Algunos ejemplos serían validar que un movimiento sea “entrada” o “salida”, impedir registrar salidas con stock insuficiente o verificar stock mínimo. Haciendo esto, los endpoints se mantienen simples y enfocados únicamente en manejar requests/responses HTTP.

Además, algunas restricciones críticas también las reforzaría a nivel de base de datos y ORM, por ejemplo con las FK, campos obligatorios, uniques o enums para tipos válidos de movimiento.

Deploy

¿Cómo deployarías este sistema? ¿En la nube, Docker, empaquetado?

Para una empresa chica, el objetivo principal del deploy es que sea económico, predecible y de bajo mantenimiento. Por eso usaría Docker para empaquetar la aplicación y la alojaría en la nube.

Al empaquetar el backend con sus dependencias exactas, el frontend y la base de datos en contenedores, se asegura que el sistema se va a ejecutar exactamente igual en cualquier lugar.

Por otro lado, lo alojaría en la nube ya que asumo que una empresa pequeña no tiene un equipo de infraestructura dedicado para mantener un servidor físico en una oficina. Esto además tiene muchos riesgos a contemplar como cortes de luz, internet, hardware que se rompe y demás.

¿Cómo manejarías las diferencias entre entorno local y producción?

La forma más común y mantenible de hacerlo es mediante variables de entorno y archivos .env. Por ejemplo, en desarrollo local usarías cierto DATABASE_URL definido en tu .env local, que va a cambiar en producción. De esta manera el mismo código funciona en ambos entornos, pero con configuraciones distintas según dónde se despliegue.

Además, nunca se deben escribir contraseñas, URLs de la base de datos, o API keys directamente en el código. Luego, el .env archivo jamás se sube a Git, por lo que siempre debe estar en el .gitignore.

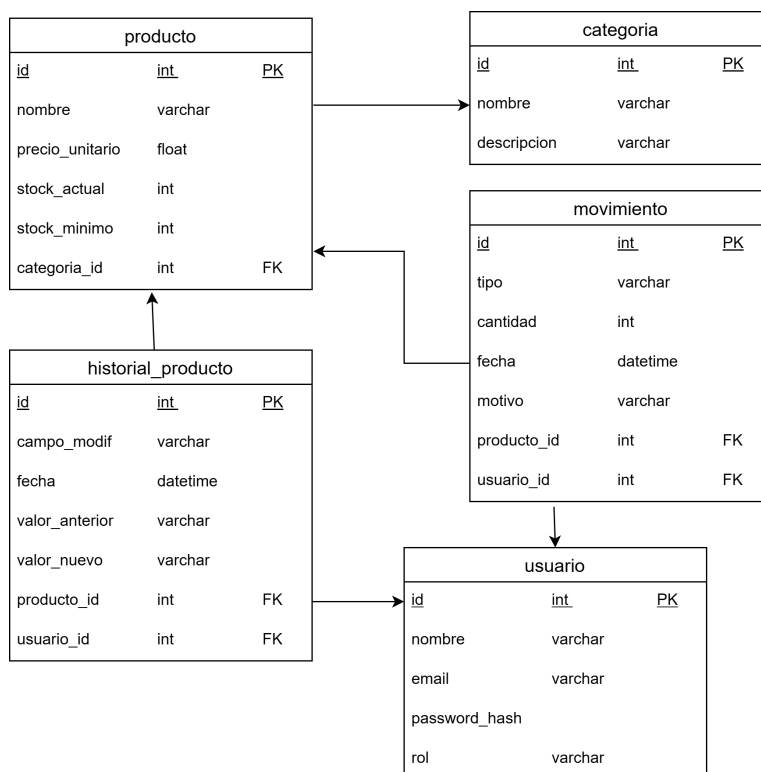
En el servidor de la nube, no se usaría el archivo .env, sino que se ingresan esos valores secretos directamente en el panel de control del proveedor de la nube o en la configuración del contenedor de Docker.

Seguridad: roles, perfiles y contraseñas

¿Cómo manejarías los distintos roles de usuario en el sistema (por ejemplo: administrador, operador de depósito, solo lectura)? ¿Dónde definirías esos permisos?

Para manejar los distintos roles, crearía en la base de datos una nueva tabla “Usuarios”. Lo haría en la misma base de datos que el resto del sistema, ya que es una aplicación relativamente pequeña y centralizada. De esta manera, cada usuario tendría un rol, y el backend verifica permisos antes de

ejecutar acciones, ya que el frontend no es seguro para controlar el acceso. Así, ya con esta tabla nueva podemos tener los movimientos de stock relacionados con el usuario activo que realizó la operación para mantener trazabilidad y auditoría.



Todo esto en lo que respecta a autorización. Por otro lado, para identificar al usuario activo, implementaría autenticación basada en JWT. El backend obtendría el usuario autenticado a partir del JWT enviado en cada request y usaría esa información para validar permisos y registrar acciones.

¿Qué consideraciones tendrías para el manejo de contraseñas? (almacenamiento, hashing, políticas mínimas.)

Algunas consideraciones básicas que tendría serían, en primer lugar, que nunca deben almacenarse en texto plano. Guardaría únicamente hashes seguros utilizando algoritmos como bcrypt o Argon2, entonces al login el sistema hasha la contraseña ingresada y compara hashes.

Además, aplicaría políticas mínimas como longitud mínima, validación básica de complejidad y uso de HTTPS para evitar exposición durante el envío. Finalmente, la validación de credenciales y generación de tokens de autenticación se manejaría completamente desde backend.

¿Cómo protegerías los endpoints según el rol del usuario autenticado?

Con JWT en cada request autenticada podemos identificar usuario y rol, lo que permite proteger los endpoints validando el rol antes de ejecutar la acción correspondiente. Intentaría centralizar la lógica de permisos para no repetir validaciones manuales en cada endpoint. En FastAPI esto podría hacerse mediante dependencias reutilizables que validen el rol del usuario antes de ejecutar la acción.

Por ejemplo, endpoints de creación o modificación solo estarían disponibles para administradores u operadores autorizados, mientras que usuarios de solo lectura podrían acceder únicamente a consultas. Vuelvo a recalcar que esta validación debería hacerse siempre en backend mediante dependencias o middleware reutilizable.

Logs del sistema

¿Qué tipo de eventos registrarías en los logs? (errores, acciones de usuario, cambios de estado, etc.)

Registraría principalmente logs relevantes de los servidores, errores de aplicación, intentos fallidos de autenticación y logs de conexión con la base de datos. Esto ayuda a detectar problemas, monitorear el comportamiento de la aplicación y facilitar el debugging y mantenimiento. Intentaría evitar logs excesivos para no generar ruido y facilitar el monitoreo de problemas reales.

¿Dónde guardarías esos logs y con qué herramienta? ¿Archivo local, servicio externo, base de datos?

Para una primera versión utilizaría logs estructurados almacenados en archivos rotativos mediante la librería estándar de logging de Python, no en la BD porque los logs crecen muchísimo y no son datos de negocio. Ya si el sistema creciera o necesitara monitoreo centralizado, podría integrarse con herramientas externas como ELK, Grafana Loki o servicios cloud de monitoreo.

¿Cómo diferenciarías los logs de aplicación de la auditoría de negocio (que ya abordaste en la Parte 1)?

Son cosas diferentes con funciones diferentes. Los logs de aplicación están orientados al monitoreo técnico y debugging del sistema. En cambio, la auditoría de negocio registra acciones importantes realizadas por los usuarios, como movimientos de stock o modificaciones de productos, con el objetivo de mantener trazabilidad e historial del sistema. Esto generalmente queda persistido a largo plazo y tiene valor funcional o legal. Por otro lado, los logs técnicos pueden rotarse o eliminarse periódicamente.

Frontend

Elección de tecnología

Tenés dos opciones: React o Streamlit. Respondé:

¿Cuál elegirías para este sistema y por qué?

Para una empresa chica como esta, necesitamos un frontend simple, prolijo, entendible, pero también funcional y que pueda escalar en caso de ser necesario. Además, ya sabemos que el sistema requiere una interfaz administrativa con múltiples vistas, tablas y formularios para actualizar información. Es por todo esto que elegiría React.

¿En qué contexto usarías Streamlit? ¿Y React? ¿Qué sacrificarías con cada uno?

Streamlit me parece muy útil para dashboards internos rápidos o prototipos orientados a datos, donde el foco esté justamente más en visualizar datos que en construir una aplicación web completa. Por otro lado, para un sistema de gestión con formularios, navegación y control de usuarios, React resulta más adecuado. Ofrece más flexibilidad para construir interfaces reutilizables y escalables, además de integrarse fácilmente con APIs REST.

Streamlit permite avanzar muy rápido, pero React da más control sobre la experiencia de usuario y la estructura de la aplicación.

Pantallas a diseñar

Vista de tabla de productos:

¿Qué columnas mostrarás? ¿Por qué esas?

Como es una lista de los productos, la idea es mostrar lo más importante de cada uno. Me centraría en la información clave que el usuario desearía ver al ingresar a la interfaz, como el "nombre", la "categoría", el "precio unitario" y el "stock actual".

Al usuario que ve esta tabla no le interesa ver datos útiles sólo para la base de datos como los ids de las PK o FK. Cabe aclarar que justamente por eso también la columna "categoría" sería mostrada con el nombre y no su id.

¿Cómo indicás visualmente si el stock está bajo? ¿Qué acciones tendrían desde esa vista?

En la tabla anterior sería conveniente una columna extra que indique el "estado" del stock de cada producto, obtenido en base a su "stock actual" y un "stock mínimo" que indique el umbral de stock bajo.

La columna podría tener dos estados: "Disponible" y "Stock Bajo". Para mayor claridad e intuición, estos estados se resaltarían en la tabla con colores Verde y Rojo, respectivamente. De esta manera, es inmediato identificar el estado de stock del producto.

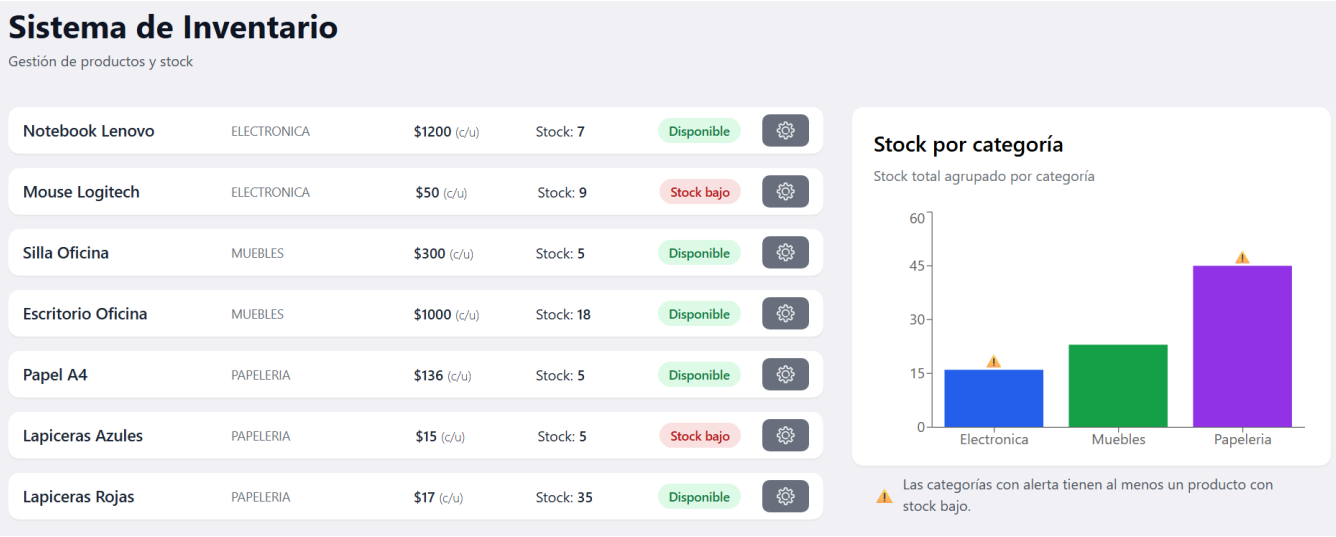
Si el usuario desea realizar alguna acción sobre el producto, sería conveniente poder acceder desde esta vista a más opciones para gestionarlo. Permitiría acciones, según el rol del usuario, como "Ver detalle", "Editar producto", "Registrar movimiento" y "Ver historial". Todas ellas serían accesibles desde el botón al final de la fila de cada producto.

Sistema de Inventario					
Gestión de productos y stock					
Notebook Lenovo	ELECTRONICA	\$1200 (c/u)	Stock: 7	Disponible	
Mouse Logitech	ELECTRONICA	\$50 (c/u)	Stock: 9	Stock bajo	
Silla Oficina	MUEBLES	\$300 (c/u)	Stock: 5	Disponible	
Escritorio Oficina	MUEBLES	\$1000 (c/u)	Stock: 18	Disponible	
Papel A4	PAPELERIA	\$136 (c/u)	Stock: 5	Disponible	
Lapiceras Azules	PAPELERIA	\$15 (c/u)	Stock: 5	Stock bajo	
Lapiceras Rojas	PAPELERIA	\$17 (c/u)	Stock: 35	Disponible	

Incorporá un gráfico de stock por producto (ej: barras comparando el stock actual de cada producto o por categoría). ¿Qué tipo de gráfico elegirías y por qué?

Considero que, como la tabla ofrece información tabular específica de cada producto, sería útil colocar a su lado un gráfico que permita una visualización más general del estado de stock de los productos *por categoría*. Entonces, por un lado la tabla o lista sirve para detalle, y por el otro el gráfico sirve para una visión rápida, siendo complementarios.

Elegiría un gráfico de barras, que es simple e intuitivo, agrupando a los productos por categoría. En el eje X tendríamos las categorías, y en el Y la cantidad de stock en total. Con él es rápido ver la cantidad de stock por categoría, y comparar estas cantidades entre sí.



Alerta de productos con stock bajo:

Diseña una vista o componente que alerte cuando un producto esté por debajo de cierto umbral de stock.

Para esta parte lo que hice fue agregar, tanto a la lista de productos como al gráfico, componentes de alerta sobre el estado de stock.

Por un lado, en la tabla de productos se agrega la columna que nombré anteriormente que indica el estado de stock con {Disponible, Stock Bajo}, resaltándose en la tabla con colores {Verde, Rojo} respectivamente.

Una vez resaltada esta información en cada producto, me pareció pertinente agregar como información de más alto nivel también al gráfico de stock por categoría un ícono de alerta o advertencia encima de las barras de las categorías con al menos un producto con stock bajo.

¿Cómo definirías ese umbral? ¿Sería fijo, configurable por categoría, configurable por producto?

Haría el umbral configurable y por producto, para poder tener un control más riguroso y que las alertas sean puedan ser más informativas. Este umbral podría usarse desde el front para comprobar fácilmente por cada producto si su `stock_actual < stock_minimo`, y si es necesario se cree la alerta.

Entonces, cuando un usuario carga un producto que se verá luego en el inventario, si existe un stock mínimo relevante, debe ingresarlo (default = 0). Esto permite que este tipo de alertas se creen solo si la información de stock mínimo es relevante para la empresa.

¿Qué cambios necesitaría el modelo de datos para soportarlo correctamente?

Se necesitaría simplemente una nueva columna en la tabla de productos, como “`stock_minimo`” para guardar el punto crítico de stock a partir del cual es necesario generar una advertencia.

¿Qué consideraciones de UX tendrías en cuenta para no generar ruido con alertas que el usuario termina ignorando?

Evitaría generar alertas excesivas o molestas, por lo que en un inicio iría por la opción de agregar la columna “Estado de stock” que mencioné antes a la vista de tabla de productos, que tenga un tamaño razonable y que más bien destaque por el color correspondiente. Priorizaría mostrar alertas relevantes y visibles solo cuando realmente requieran acción (es por eso que si no se ingresa un stock mínimo para el producto este por default sería = 0).

Formulario para registrar un movimiento (entrada o salida):

¿Qué campos incluís?

Como mencioné anteriormente, permitiría ingresar a la opción “Registrar Movimiento” como una de las opciones disponibles desde el botón al final de cada fila de producto. Como aclaración, señalo que sería indispensable implementar una búsqueda por nombre o id de producto para filtrar y que se actualice la lista. De esa manera se podría acceder más rápido a lo que se busca (en este caso no se solicitaba y por ello no lo agregué en el código, pero debería implementarse si se sigue esta lógica del botón).

Luego, en la ventana que se abra al seleccionar la opción “Registrar Movimiento” se vería el formulario. En ella deberían incluirse los campos:

- Producto (automático, ya que ingresamos desde el mismo)
- Tipo de movimiento (entrada/salida)
- Cantidad
- Motivo
- Usuario (automático según sesión)
- Fecha (automática por default, o configurable si el usuario desea registrar un movimiento pasado)

En el código, para simplificar, lo implementé haciendo que directamente al apretar al botón del producto, este se seleccione y se pase directamente al formulario de “Registrar Movimiento”.

The image shows a modal form titled "Registrar movimiento" overlaid on a background interface. The background includes a table of products with prices (e.g., \$120, \$50, \$30, \$100, \$136, \$15, \$17) and a bar chart on the right with a y-axis from 0 to 60. The form itself contains the following fields:

- Producto seleccionado:** Notebook Lenovo
- Tipo de movimiento:** A dropdown menu with "Entrada" selected.
- Cantidad:** A text input field containing the number "1".
- Motivo:** A large text area for notes.
- Buttons:** "Cancelar" and "Guardar movimiento".

Esta es una forma de hacerlo. Otra podría ser tener un botón aparte, independiente de la lista de productos, llamado “Registrar Movimiento” que permita hacer una carga de los productos desde allí. En ese caso el campo de producto no sería automático, sino que debería seleccionarse desde por ejemplo una lista desplegable.

¿Cómo validás que no quede stock negativo antes de enviar?

Validaría en frontend cuando el usuario intente guardar para dar feedback inmediato, lo cual mejora la experiencia, pero la validación final debe hacerse también en backend para garantizar la integridad de los datos.

¿Qué botones ponés y dónde? Defendé esa decisión.

Lo mantendría super simple y claro, pondría un botón principal de “Guardar movimiento” destacado visualmente y un botón secundario de “Cancelar”. Los ubicaría al final del formulario siguiendo el flujo natural de lectura.

Registrar movimiento

Producto seleccionado
Notebook Lenovo

Tipo de movimiento
Salida

Cantidad
8

No hay stock suficiente para realizar esta salida (Stock actual: 7)

Motivo

Git

Se utilizó un flujo con ramas efímeras para cada feature, integrando los cambios mediante merge commits para mantener la trazabilidad.

Primero tenemos el **main**, donde comencé el proyecto. Luego creé una rama **develop** desde la cual creé 2 más:

1. Una para desarrollar el frontend con el backend funcionando con la base de datos mock.
 - Cuando terminé esta versión lo que hice fue hacer un merge con la rama develop
2. Otra rama saliendo de develop para cambiar a que el backend funcione con la base de datos real basada en los modelos ORM desarrollados. Esta todavía no tuvo un merge con develop.

Uso de IA

¿Qué herramienta usaste?

A este proyecto lo comencé sólo con conocimientos sobre desarrollo backend con FastAPI, y sin conocimientos de frontend con React. Fue una oportunidad para mí de familiarizarme con conceptos nuevos, es por ello que el uso de IA fue un gran apoyo para comprender y avanzar más rápido.

Usé principalmente ChatGPT en modo chat web. No usé herramientas más avanzadas como Claude ya que quería que el proyecto me sirva para aprender y ganar experiencia, por lo que fuí desarrollando los componentes parte por parte preguntando al chat sobre los conceptos importantes a tener en cuenta, lo que me permitió tomar apuntes.

¿Para qué la usaste puntualmente?

La usé primero para discutir los requerimientos que identifiqué a partir del PDF con la consigna. Por ejemplo, para la elección de la tecnología del backend estaba segura de avanzar con Python + FastAPI pero para la parte del front me ayudó a analizar las ventajas y desventajas de Streamlit y React.

Una vez identificado esto, comencé a buscar tutoriales en Youtube sobre cómo conectar estos componentes, ya que quería comprender realmente las bases de lo que estaba realizando, algo que sólo usando IA podría saltarme y seguir avanzando. Encontré este recurso que fue de mucha ayuda:

<https://youtu.be/1vDdFoorA74?si=z5qb6ou5upHsmcpu>

Una vez que entendí el flujo de la información de la conexión front-back, comencé con el desarrollo del backend. Una vez lista la versión del endpoint con la BD mock, ChatGPT me resultó muy útil para agilizar el diseño del frontend, ya que es muy bueno traduciendo consignas en lenguaje natural como “Quiero que las columnas de productos sean más pequeñas verticalmente” a las modificaciones en el código. Haciendo las preguntas correctas, pude aprender luego como hacer estos cambios yo misma con, por ejemplo, las clases de Tailwind, y los conceptos más importantes de React.

Por otro lado, también la usé para validar mis respuestas a las preguntas a responder en el PDF, como una instancia extra de aprendizaje y corrección.

¿Hubo algo que te sugirió y decidiste no usar o cambiar? ¿Por qué?

Al comenzar este take home, como nunca había desarrollado un proyecto fullstack de este estilo, usé Claude en modo planning para que me propusiera una posible estructura general del proyecto a partir de un prompt que previamente armé con ayuda de ChatGPT, basado en toda la discusión de requerimientos y decisiones técnicas que había trabajado previamente.

Sin embargo, esa propuesta inicial incluía bastante más complejidad y funcionalidades de las que realmente se pedían para esta evaluación, por lo que decidí no utilizarla directamente como base del proyecto. Preferí tomarme el tiempo de construir la solución por mi cuenta e ir comprendiendo realmente cada parte de la implementación, aprovechando la oportunidad para aprender las bases y entender en profundidad todo lo que estaba desarrollando.

Aun así, ese proceso inicial me resultó útil para tener una visión general de cómo suele estructurarse un proyecto de este tipo y cuáles son algunas de las responsabilidades y capas que normalmente lo componen.